Hardware Decides the Configuration: A CPU-versus-GPU Benchmark of a Retrieval-Augmented Chatbot on Philippine Consumer-Remedy Law

Mark Emmanuel P. Sagarino
College of Computer Studies
Jose Rizal University
Mandaluyong City, Philippines
markemmanuel.sagarino@my.jru.edu

Ram Kelly Jerzen Leliza
College of Computer Studies
Jose Rizal University
Mandaluyong City, Philippines
ramkellyjerzen.leliza@my.jru.edu

Sean John C. Dizon
College of Computer Studies
Jose Rizal University
Mandaluyong City, Philippines
seanjohn.dizon@my.jru.edu

***Abstract*—Five indexing configurations of a retrieval-augmented chatbot over 73,652 characters of Philippine consumer-remedy law were measured on two Colab runtimes, a shared CPU and a Tesla T4, with every other variable held fixed. Retrieval metrics are bit-identical across the two devices; only generation differs. The GPU is 51.4 times faster end to end on the benchmark subset, but the figure is not uniform: the generation speedup grows with chunk size, from 26.2x at 400 characters to 81.8x at 1,200, because prompt length is what the CPU cannot absorb and what the GPU parallelises. Normalised per output token, the cost of moving from 400- to 1,200-character chunks is 4.06x on the CPU and 1.30x on the GPU. The practical consequence is that the configuration with the best retrieval quality, 1,200 characters with 150 of overlap, takes 304.90 s per answer on the CPU and 6.31 s on the GPU: the same parameters are unusable on one device and comfortable on the other. Parameter selection and hardware selection are therefore not separable decisions.**

Keywords—retrieval-augmented generation; CPU-GPU benchmarking; chunk size; execution time; speedup; Chroma; Qwen2.5; Tesla T4.

I. Experimental Setup

A. Corpus and Test Set

The knowledge base is five plain-text documents totalling 73,652 characters: Republic Act No. 7394, the Consumer Act [1]; No. 8792, the Electronic Commerce Act [2]; No. 10642, the Philippine Lemon Law [3]; No. 11967, the Internet Transactions Act [4]; and the Department of Trade and Industry's published consumer guidance [5]. Nine questions were used, three at each difficulty level, with nineteen evidence anchors fixed before any measurement was taken. Because chunk identifiers change whenever the chunk size changes, an anchor is not a chunk identifier but a short verbatim string: the minimum span of corpus text that carries the fact. Ground truth is therefore configuration-independent. A tenth question with no supporting evidence serves as a guardrail and is excluded from the metrics, which are undefined when no relevant passage exists.

B. Pipeline and Fixed Components

Documents were split with a recursive character splitter, embedded with all-MiniLM-L6-v2 into 384-dimensional vectors [6], and indexed in an in-memory Chroma collection built fresh for every configuration, with the stored vector count asserted against the chunk count so that a stale index halts the run rather than producing quietly wrong numbers. Generation used Qwen2.5-1.5B-Instruct [7] with a 160-token ceiling and sampling disabled: a sampled answer cannot be compared across devices. A locally executed model was essential to the exercise. A hosted API model would run on the provider's hardware, and the local runtime setting would make no difference to its timings.

C. Devices and Instrumentation

Two Colab sessions were used. Each detects the active device and writes that label into every result row, so the two sets of files merge afterwards without either being mistaken for the other. Section 9 of the notebook reports where the weights actually sit rather than what was requested, because selecting a GPU runtime does not by itself place every operation on it.

Timers are separated. Indexing covers splitting, embedding and index construction; retrieval covers the retriever call alone; generation covers the model call alone. CUDA work is asynchronous, so the generation timer is stopped only after an explicit device synchronisation; without it the GPU figures would be the time taken to queue the work rather than to do it. Model loading is timed separately and reported as a one-time setup cost, never folded into a per-question measurement.

TABLE I. Experimental Environment

Setting	Value
Corpus	5 documents, 73,652 characters
Test set	9 scored questions, 19 evidence anchors, 1 guardrail
Embedding model	all-MiniLM-L6-v2, 384 dimensions
Vector store	Chroma, in-memory, rebuilt per configuration
Language model	Qwen2.5-1.5B-Instruct, bfloat16
Decoding	greedy, max 160 new tokens, sampling off
CPU runtime	Colab shared CPU; model load 71.24 s
GPU runtime	Tesla T4, 14.6 GiB; model load 40.99 s
Repetitions	3 timed, each preceded by a discarded warm-up

II. Configurations Tested

Part A swept five configurations across all nine questions. Runs A, B and C vary chunk size at a constant 12.5 per cent overlap ratio, which isolates size. Run D doubles the overlap of Run B at the same size, which isolates overlap. Run E raises retrieval depth from three to five at the same chunking as Run B, which isolates depth. Part B re-ran the three size variants on a single question with three timed repetitions each.

Retrieval results are identical on both devices: Recall@k, Precision@k and the ranked chunk identities match for all five configurations. This is a useful control: it establishes

that the embedding and search path is numerically stable across hardware, and therefore that any divergence observed later is attributable to generation alone.

TABLE II. PARAMETER SWEEP, NINE QUESTIONS PER CONFIGURATION

Run	Size	Ovl	k	Chunks	R@k	P@k	CPU s	GPU s
A	400	50	3	268	0.389	0.148	185.94	6.08
B	800	100	3	125	0.444	0.185	220.37	4.98
C	**1200**	**150**	**3**	**78**	**0.500**	**0.222**	**304.90**	**6.31**
D	800	200	3	131	0.389	0.148	193.54	4.39
E	800	100	5	125	0.444	0.111	321.58	4.82

a. Times are the mean total response per question. R@k and P@k are identical on both devices.

Two of the three parameter effects are negative. Doubling the overlap from 100 to 200 characters, Run B to Run D, lowers Recall@k from 0.444 to 0.389 and Precision@k from 0.185 to 0.148. The additional overlap produces near-duplicate neighbouring chunks that compete for the same three slots, so a second genuinely distinct passage is displaced by a restatement of the first. Raising retrieval depth from three to five, Run B to Run E, returns exactly the same Recall@k of 0.444 while halving Precision@k from 0.185 to 0.111: the two extra chunks contained no evidence that the first three did not already contain, and on the CPU they cost an additional 46 per cent of response time. Only chunk size improved retrieval, and it did so monotonically across A, B and C.

III. CPU versus GPU Execution Time

Table III reports the mean of three timed repetitions per configuration on each device. No repetition deviates from its own configuration mean by more than 1.4 per cent on either device, which is tight enough that the ratios in Table IV are not artefacts of a noisy session.

TABLE III. AVERAGE EXECUTION TIME BY DEVICE

Device	Run	Index s	Retr ms	Gen s	Total s	Tokens
CPU	A	19.489	40.7	180.687	180.730	160
CPU	B	15.743	41.5	260.682	260.727	154
CPU	C	10.084	46.3	302.797	302.848	66
GPU	A	0.625	10.1	6.892	6.904	160
GPU	B	0.427	9.3	5.586	5.598	122
GPU	C	0.349	8.9	3.702	3.714	66

TABLE IV. GPU SPEEDUP, COMPUTED AS CPU TIME DIVIDED BY GPU TIME

Run	Indexing	Retrieval	Generation	Total	Per token
A	31.18x	4.03x	26.22x	26.18x	26.22x
B	36.87x	4.46x	46.67x	46.58x	36.97x
C	28.89x	5.20x	81.80x	81.54x	81.80x
Mean	**32.31x**	**4.56x**	**51.56x**	**51.43x**	**48.33x**

b. Model loading, a one-time setup cost, improved separately from 71.24 s to 40.99 s, a 1.74x gain. Run B generated 154 tokens on the CPU and 122 on the GPU, so its per-token figure is the honest one; A and C produced equal token counts on both devices.

A. The Speedup Is Not a Single Number

The three components behave differently, and reporting one blended figure would conceal the most informative result. Retrieval gains only 4.56x. That is the expected outcome rather than a disappointment: Chroma's similarity search executes on the host CPU in both sessions, so the entire gain comes from embedding the query vector, which is the one part of retrieval that touches the accelerator. Indexing gains 32.31x, since embedding the whole corpus is exactly the dense, batched arithmetic a GPU exists for. Generation gains 51.56x and carries the end-to-end figure, because generation is at least 99.6 per cent of total response time in every configuration on both devices.

B. Why the Gain Grows with Chunk Size

The generation speedup is not constant across configurations. It rises from 26.22x at 400 characters to 46.67x at 800 and 81.80x at 1,200 — a factor of three across the sweep. Normalising by output token isolates the cause. On the CPU, the cost of producing one token rises from 1.129 s at Run A to 4.588 s at Run C, a penalty of 4.06x for the longer prompt. On the GPU the same change costs 0.0431 s to 0.0561 s, a penalty of only 1.30x. Retrieval depth is three in all three runs, so the number of chunks in the prompt is constant; what changes is their length, and therefore the length of the context the model must read before it writes anything. That prefill pass is sequential arithmetic on the CPU and parallel arithmetic on the GPU. The CPU pays for prompt length almost linearly; the GPU very nearly absorbs it.

The consequence is visible in the raw wall-clock ordering. On the CPU, Run C is the slowest of the three size variants at 302.80 s of generation. On the GPU it is the fastest at 3.70 s. The ranking of configurations by response time inverts between devices. A team that selected parameters on CPU timings and then deployed on a GPU would have optimised against the wrong cost model.

IV. Answer Quality

Answer quality is the fraction of required facts from the answer key that appear in the generated answer, measured over the three-question generation subset. It tracks chunk size in the same direction as retrieval: 0.667 at Run A, 0.500 at Run B and 1.000 at Run C. Run B is the anomaly. It retrieves better than Run A on both metrics yet answers worse, which is the clearest evidence in this experiment that retrieving a relevant chunk and using it are separate successes. Its 800-character chunks are long enough to displace a competing passage from the top three but, on the questions in the subset, still too short to carry a complete statutory provision, so the model receives a fragment and completes it imperfectly.

One configuration scored differently on the two devices. Run E rose from 0.833 on the CPU to 1.000 on the GPU, and in Part B the same prompt produced 154 output tokens on the CPU and 122 on the GPU. Sampling was disabled in both sessions, so this is not randomness in the decoding rule. Greedy decoding is deterministic in exact arithmetic, but the arithmetic is not exact: different kernels and reduction orders on the two devices perturb the logits slightly, and where two candidate tokens are nearly tied the argmax can fall differently. From that point Reproducibility of a generated answer is therefore a property of the hardware as well as of the decoding configuration.

The guardrail question, which asks about a named online marketplace's refund window, is answerable from no document in the corpus. On both devices the chatbot declined, stating that the context contains no fixed number of days, rather than assembling a plausible figure from the nearest retrieved passages. Over-refusal was checked too: all nine scored questions were attempted, so the decline is selective.

V. Recommended Configuration

The selection rule applied in the notebook ranks configurations on a weighted quality score, keeps everything within 0.05 of the best, and settles the remainder on speed.

On the CPU that rule returns Run C alone. On the GPU it admits Run E as well, because Run E's answer quality rose to 1.000 on that device, and then prefers Run E because it is faster.

We adopt Run C rather than Run E. Run E enters the tie only by way of the single score that was shown in the previous section to be device-dependent; on the CPU the same configuration scores 0.833 and does not qualify. Run C scores 1.000 on both devices, holds the best Recall@k and the best Precision@k in the sweep, and produces the fewest chunks, so its index is the cheapest to build and to hold. Run E also carries the worst Precision@k of any configuration tested, 0.111. A recommendation that depends on which device happened to run it is not a recommendation.

The device recommendation is the GPU, and it is not marginal. At Run C the CPU needs 304.90 s per answer; the T4 needs 6.31 s. No plausible tolerance for latency places a five-minute answer inside a conversational interface. The honest summary of this experiment is that the parameter sweep identified the best configuration and the CPU made it unusable; the GPU is what converts the measured quality advantage into a usable one.

TABLE V. RECOMMENDED OPERATING POINT

Setting	Value
Configuration	Run C
Chunk size	1,200 characters
Chunk overlap	150 characters (12.5 per cent)
Retrieval depth	k = 3
Resulting chunks	78
Recall@k / Precision@k	0.500 / 0.222 (best in sweep)
Answer quality	1.000 on both devices
Device	GPU (Tesla T4)
Response time	6.31 s, against 304.90 s on CPU

VI. LIMITATIONS

The Tesla T4 is a Turing device and has no native bfloat16 arithmetic; bfloat16 runs there through emulation. Both sessions were nevertheless run in bfloat16, which keeps hardware as the single changed variable and makes the comparison clean, at the cost of measuring the T4 below its capability. Every speedup reported here is therefore a lower bound. A float16 run on the same card would be expected to widen the gap, and the size of that effect was not measured.

Part B rests on one question repeated three times; it is a timing benchmark, and its answer-quality column is not a quality result. The test set is small at nine questions and nineteen anchors — enough to separate configurations that differ substantially, not enough to resolve close ones.

Each device was measured in a single Colab session. Within-session repetition was controlled with warm-ups and three timed runs, but between-session variance on shared infrastructure was not estimated, and the CPU allocation in particular is not guaranteed to be identical from one session to the next.

A confound affects Run C specifically. The embedding model truncates its input at 256 word-pieces, roughly 1,000 characters, while the full chunk is passed to the language model. At 1,200 characters a chunk is therefore embedded only in part and retrieved in full. Some of the advantage attributed to the larger chunk is the extra text reaching the model rather than a better vector representation of it, and the present design cannot separate the two.

Finally, as section IV established, greedy decoding was not bitwise reproducible across devices. Any cross-device comparison of generated text, including the per-token figures in Table IV, carries that uncertainty.

VII. POSSIBLE IMPROVEMENTS

Four changes would address the limitations above directly. Running the T4 in float16 and comparing it against the bfloat16 run on the same card would quantify the emulation penalty and convert the lower bound reported here into an interval. Fixing the output length, by setting a minimum equal to the maximum new-token count, would remove token count as a confound in the wall-clock speedups and make Run B's raw and per-token figures coincide. Replacing the embedding model with one whose context window reaches 512 tokens would remove the truncation confound at 1,200 characters and allow larger chunks to be evaluated honestly. Repeating each device in two or three independent sessions would put an error bar on the ratios rather than a single point estimate.

Two further changes address deployment rather than measurement. Batched generation would let the GPU answer several questions concurrently, raising throughput well beyond the 51x reported here. For the opposite constraint, four-bit or eight-bit quantisation would be the first thing to try if CPU deployment were required, since the measured CPU cost is dominated by dense arithmetic over full-precision weights.

A final methodological note. Because the ranking of configurations by speed inverts between devices, a sweep run on a development laptop would recommend differently from one run on the deployment hardware. The sweep and the device choice are a single joint decision.

AUTHOR CONTRIBUTIONS

Emmanuel Sagarino designed the experiment and carried out the technical work. This covers the assembly and verification of the five-document corpus, the authoring of the nine test questions and their nineteen verbatim evidence anchors, the implementation of the notebook including the timing instrumentation, the device-tagging and merge logic and the scoring functions, the execution of both the CPU and the GPU sessions, the analysis of the merged results, and the drafting of this report.

Leliza and Dizon reviewed the draft report and the notebook output and provided feedback on the presentation of the results.

REFERENCES

[1] Republic of the Philippines, Republic Act No. 7394, "The Consumer Act of the Philippines," Apr. 13, 1992.

[2] Republic of the Philippines, Republic Act No. 8792, "Electronic Commerce Act of 2000," Jun. 14, 2000.

[3] Republic of the Philippines, Republic Act No. 10642, "Philippine Lemon Law," Jul. 15, 2014.

[4] Republic of the Philippines, Republic Act No. 11967, "Internet Transactions Act of 2023," Dec. 5, 2023.

[5] Department of Trade and Industry, "FAQs," DTI E-Commerce Philippines, and Fair Trade Enforcement Bureau, "How to file a consumer complaint?" [Online]. Available: ecommerce.dti.gov.ph, fairtrade.dti.gov.ph

[6] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence embeddings using Siamese BERT-networks," in Proc. EMNLP-IJCNLP, 2019.

[7] A. Yang et al., "Qwen2.5 technical report," arXiv:2412.15115, 2024.

[8] P. Lewis et al., "Retrieval-augmented generation for knowledge-intensive NLP tasks," in Proc. NeurIPS, 2020.